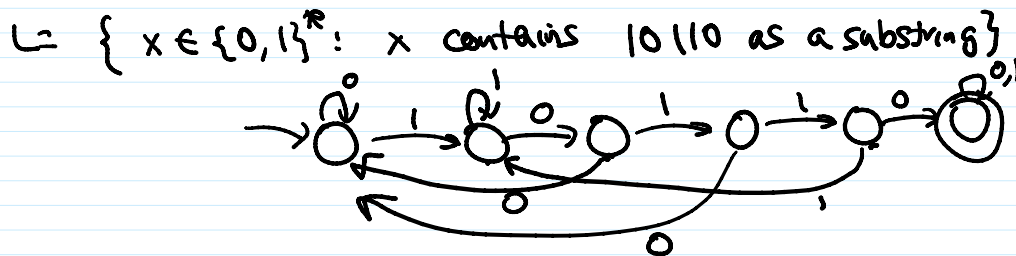


Problem Given strings $u = a_1 a_2 \dots a_n \in \Sigma^*$ "text"
 $v = b_1 b_2 \dots b_m \in \Sigma^*$ "pattern"
 is v a substring of u ? $m \leq n$

e.g. $u = 01101011010$
 $v = 10110$

brute force: $O(mn)$ worst-case

"DFA method": $O(f(m) + n)$ time
 build DFA $(f(m) = O(|\Sigma|m))$ not obvious



Knuth-Morris-Pratt '77 $O(n)$ time
 using "compressed" representation of DFA
 with tricky details

Rabin-Karp '87 $O(n)$ time rand.
 simpler (general)

Digression ...

Subproblem Alice has a string $u \in \{0,1\}^*$
 Bob has a string $v \in \{0,1\}^*$ ($|u|=|v|=n$)
 Want to test whether $u=v$.

Goal - minimize communication complexity
 # bits transmitted

Obvious idea - need to transmit n bits

another idea - "checksum"
 (log n bits)

Obvious idea - need to transmit n bits

another idea - "checksum"
of 1's ($\log n$ bits)

Wrong! (countered 0110, 0101)

or use another mapping $f: \{0,1\}^* \rightarrow \{0,1,\dots,p-1\}$
e.g. for small p .

for any f , $\exists$ counterex: ✗

randomized idea -

fix prime $p > n$.
pick rand base x .

let $F_x: \{0,1\}^* \rightarrow \{0,1,\dots,p-1\}$

Called
fingerprint
function →

$$F_x(\underbrace{a_{n-1}a_{n-2}\dots a_0}) = \left(\sum_{i=0}^{n-1} a_i x^i \right) \bmod p$$

Monte Carlo Alg'm:

$x = \text{rand}(0, p-1)$ ($\leftarrow 4x$)

#bits transmitted

✓ = $O(\log p)$

Alice transmits $F_x(u)$ to Bob

Bob says "probably equal" if $F_x(u) = F_x(v)$
"definitely not equal" else

Error Analysis:

if $u = v$, correct

if $u \neq v$, say $u = a_{n-1}\dots a_0$
 $v = b_{n-1}\dots b_0$

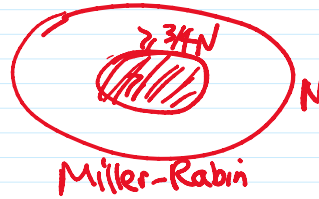
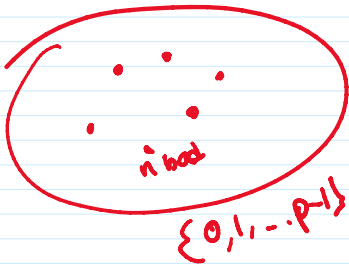
alg'm errs $\Leftrightarrow F_x(u) = F_x(v)$

$$\Leftrightarrow \sum_{i=0}^{n-1} a_i x^i \equiv \sum_{i=0}^{n-1} b_i x^i \bmod p$$

$$\Leftrightarrow \sum_{i=0}^{n-1} (a_i - b_i) x^i \equiv 0 \bmod p$$

Counting Lemma A nonzero polynomial of $\deg \leq n-1$
has $\leq n-1$ roots, mod p .

$$\Rightarrow \Pr[\text{alg'm errs}] \leq \frac{n-1}{p}$$



Pick $p \approx n^{d+1} \Rightarrow$ prob err $O(\frac{1}{n^d})$
 # bits transmitted $\log p$
 $= O(d \log n)$

Back to string matching problem...

Rabin-Karp Rand Alg'm (Monte Carlo)

$x = \text{rand}(0, p-1)$

$B = F_x(b_1 b_2 \dots b_m)$

$A = F_x(a_1 a_2 \dots a_m)$

for $i = 0$ to $n-m$ do

// go from $F_x(a_{i+1} a_{i+2} \dots a_{i+m})$
 to $F_x(a_{i+2} a_{i+3} \dots a_{i+m+1})$

$$A = \left((A - a_{i+1} x^{m-1}) x + a_{i+m+1} \right) \bmod p$$

$O(1)$ time
 in $(\log p)$ -bit RAM

if $A = B$
 return "possible match at i "

return "definitely no match"

Runtime: $O(n)$ arithmetic ops on $(\log p)$ -bit #
 $= O(d \log n)$

Error Analysis: let $E_i = [\text{alg'm errs at } i^{\text{th}} \text{ iter}]$

by Alice-Bob, $\Pr(E_i) \leq \frac{m-1}{p}$

$$\begin{aligned}\Pr[\text{alg'm errs}] &= \Pr\left(\bigcup_{i=0}^{n-m} E_i\right) \\ &\leq \sum_{i=0}^{n-m} \Pr(E_i) \\ &\leq \frac{n-m}{p} \leq \frac{n^2}{p}\end{aligned}$$

pick $p \approx n^{d+2} \Rightarrow$ error prob $O\left(\frac{1}{n^d}\right)$

Las Vegas Version:

1. run Monte Carlo version of Karp-Rabin
2. if it says "possible match at i " {
3. verify $a_{i+1} \dots a_{i+m} = b_1 \dots b_m$
4. if so, return "match at i "
5. else run brute force
- }

Always correct

Runtime analysis:

line 3 takes $O(m)$ time

line 5 takes $O(mn)$ time $\leftarrow$ prob $O\left(\frac{1}{n^d}\right)$

$$\text{expected runtime} = O(n + \cancel{m} + mn \cdot \frac{1}{n^d})$$

$$= \boxed{O(n)}$$

e.g. by setting $d=1$

$$\begin{aligned}&n \cdot \left(1 - O\left(\frac{1}{n^d}\right)\right) \\ &+ mn \cdot O\left(\frac{1}{n^d}\right)\end{aligned}$$